SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

Dr. M. Anoop

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario based

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2

- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

1. Most suitable scheduling algorithm

Priority Scheduling

Priority order:

- P1 – Payment gateway transactions → High Priority
- P2 – Seat availability checks → Medium Priority
- P3 – Ticket confirmation and printing → Low Priority

2. Justification

Priority Scheduling ensures that the most critical tasks are executed first. In a railway ticket booking system, payment transactions should be completed immediately to avoid transaction failures. After successful payment, seat availability is checked, followed by ticket confirmation and printing. This reduces the response time for important operations, improves CPU utilization by executing urgent processes without unnecessary delays, and provides better customer service during heavy traffic.

3. Sample execution order

Example Burst Times:

Process	Burst Time	Priority
P1	5 ms	High
P2	8 ms	Medium
P3	15 ms	Low

Execution Order:

P1 → P2 → P3

If multiple processes are present:

P1 → P1 → P2 → P2 → P3 → P3

4. Drawback

Priority Scheduling may cause **starvation**, where low-priority processes such as ticket confirmation keep waiting if high-priority requests continuously arrive.

Question 2 – Deadlock in a Banking Transaction Server

1. Resource Allocation Graph (Text-Based)

Resources:

A = Account Database

B = Customer Info Database

C = Transaction Logs\

A → T1 → B

B → T2 → C

C → T3 → A

2. Has deadlock occurred?

Yes.

Reason:

- T1 holds A and waits for B.
- T2 holds B and waits for C.
- T3 holds C and waits for A.

This forms a circular wait:

T1 → T2 → T3 → T1

Since every transaction is waiting for a resource held by another transaction, the system is in a deadlock.

3. Recommended technique

Deadlock Prevention

A fixed resource allocation order should be followed.

Example:

A → B → C

Every transaction must request resources only in this order. This removes the circular wait condition and prevents deadlock.

4. Redesign locking order

Current:

T1 : A → B

T2 : B → C

T3 : C → A

Redesigned:

T1 : A → B → C

T2 : A → B → C

T3 : A → B → C

Using the same locking order for all transactions prevents future deadlocks.

Question 3 – Mobile OS App Switching Lag

1. Why does the CPU remain idle?

Although many applications are active, most of them are waiting for I/O operations such as disk access, network communication, or user input. Since they are not ready to execute, the CPU has no ready process to schedule and remains idle.

2. Is the time quantum helping or hurting performance?

The current time quantum of **20 ms** is hurting performance.

A larger quantum causes interactive applications to wait longer before getting CPU time, making app switching slower and increasing response time.

3. Better scheduling recommendation

Use Multilevel Feedback Queue (MLFQ) or reduce the Round Robin time quantum to 10 ms.

This gives interactive applications quicker access to the CPU while background tasks continue with lower priority.

4. How will this improve performance?

Reducing the quantum or using MLFQ:

- Improves app responsiveness.
- Reduces app switching delay.
- Increases system throughput.
- Improves CPU utilization.
- Provides a smoother user experience.

Question 4 – Deadlock Risk in an Operating System Update

1. Can this lead to a deadlock?

Yes.

The given resource allocation can result in a deadlock.

2. Circular wait condition

M1 holds R1 and waits for R2.

M2 holds R2 and waits for R3.

M3 holds R3 and waits for R1.

Circular wait:

M1 → M2 → M3 → M1

Each module waits for a resource held by another module, causing deadlock.

3. Banker's Algorithm style solution

Before allocating a resource, the operating system checks whether granting the request keeps the system in a **safe state**.

If the request leads to an unsafe state, it is delayed until enough resources become available.

Safe execution sequence:

M1 → M2 → M3 → M4

Each module releases its resources before the next module proceeds, preventing deadlock.

4. Prioritizing modules

Suggested priority:

1. M1 – Copy update files
2. M2 – Install drivers
3. M3 – Update registry
4. M4 – Validate system integrity

This allows essential update tasks to complete first, reducing waiting time and improving CPU utilization.

Question 5 – Cloud Server CPU Utilization Analysis

Given:

- I/O wait probability (**p**) = **0.8**
- Number of VMs (**n**) = **7**

Formula:

$$\text{CPU Utilization} = 1 - p^n$$

1. Calculate CPU utilization

$$= 1 - (0.8)7$$

$$= 1 - 0.2097$$

$$= 0.7903$$

$$\text{CPU Utilization} \approx 79.03\%$$

2. Interpretation

A CPU utilization of **79.03%** indicates that the server is well utilized, but about **21%** of CPU capacity is still idle. This means there is room for improving resource utilization.

3. Effect of changing the number of VMs

- **Increasing the number of VMs** increases throughput because the CPU can execute other VMs while some are waiting for I/O. However, too many VMs can increase context switching and reduce performance.
- **Decreasing the number of VMs** reduces throughput because fewer processes are available to use the CPU, leading to more idle time.

4. Two OS-level strategies to improve CPU utilization

1. **Use an efficient CPU scheduling algorithm** such as Multilevel Feedback Queue (MLFQ) or an optimized Round Robin scheduler to improve process execution and reduce waiting time.
2. **Overlap CPU execution with I/O operations** using asynchronous I/O or multithreading so that while one VM waits for network or disk operations, another VM can utilize the CPU effectively.